3_modeling

2025-06-13

Modeling

```
knitr::opts_chunk$set(echo = TRUE)
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(here)
```

```
## here() starts at /home/rstudio/Documents/ads503_project_g1
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
library(pls)
```

```
##
## Attaching package: 'pls'
##
## The following object is masked from 'package:caret':
```

```
##
##     R2
##
## The following object is masked from 'package:stats':
##
##     loadings
```

```
library(gt)
```

Read and Prepare Data

```
# Read in modeling dataset
df_model <- readRDS(here("Data/Clean", "heart_disease_modeling.rds"))
# Ensure factor levels for binary outcome
df_model$has_disease <- factor(df_model$has_disease, levels = c("No", "Yes"))

# Set seed for reproducibility
seed <- 13579
set.seed(seed)
```

Train/Test Split

```
train_index <- createDataPartition(df_model$has_disease, p = 0.8, list = FALSE)
train_data  <- df_model[train_index, ]
test_data   <- df_model[-train_index, ]
```

Global Model Settings

```
# Performance metric and resampling
global_metric <- "ROC"
ctrl <- trainControl(
  method          = "cv",
  number          = 5,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  verboseIter     = FALSE
)
# Preprocessing strategy
process_strat <- c("center", "scale")

# Tuning grids
#knn
K_grid <- data.frame(k = seq(1, 25,
                             by = 2))
#plsda
pls_grid <- expand.grid(ncomp = 1:10)
#penalized
#lasso
lasso_grid <- expand.grid(alpha = 1,
                          lambda = 10^seq(-4, 0, length = 50))
#ridge
ridge_grid <- expand.grid(alpha = 0,
                          lambda = 10^seq(-4, 0, length = 50))
```

```r
#elasticnet
enet_grid <- expand.grid(alpha = seq(0.1, 0.9, by = 0.1),
                         lambda = 10^seq(-4, 0, length = 50))
```

Initialize Results Table

```r
#empty results table of performance metrics
results_tbl <- tibble(
  Model       = character(),
  Accuracy    = numeric(),
  Sensitivity = numeric(),
  Specificity = numeric(),
  ROC_AUC     = numeric()
)
```

1. Logistic Regression

```r
#set seed
set.seed(seed)
#train logistic model
glm_model <- train(
  has_disease ~ .,
  data       = train_data,
  method     = "glm",
  family     = "binomial",
  metric     = global_metric,
  trControl  = ctrl,
  preProcess = process_strat
)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```r
# Testing set performance
glm_preds      <- predict(glm_model, test_data)
glm_probs      <- predict(glm_model, test_data, type = "prob")[, "Yes"]
roc_glm        <- roc(test_data$has_disease, glm_probs)
```

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

```r
glm_cm_test    <- confusionMatrix(glm_preds, test_data$has_disease,
                                  positive = "Yes")

# Training set performance
```

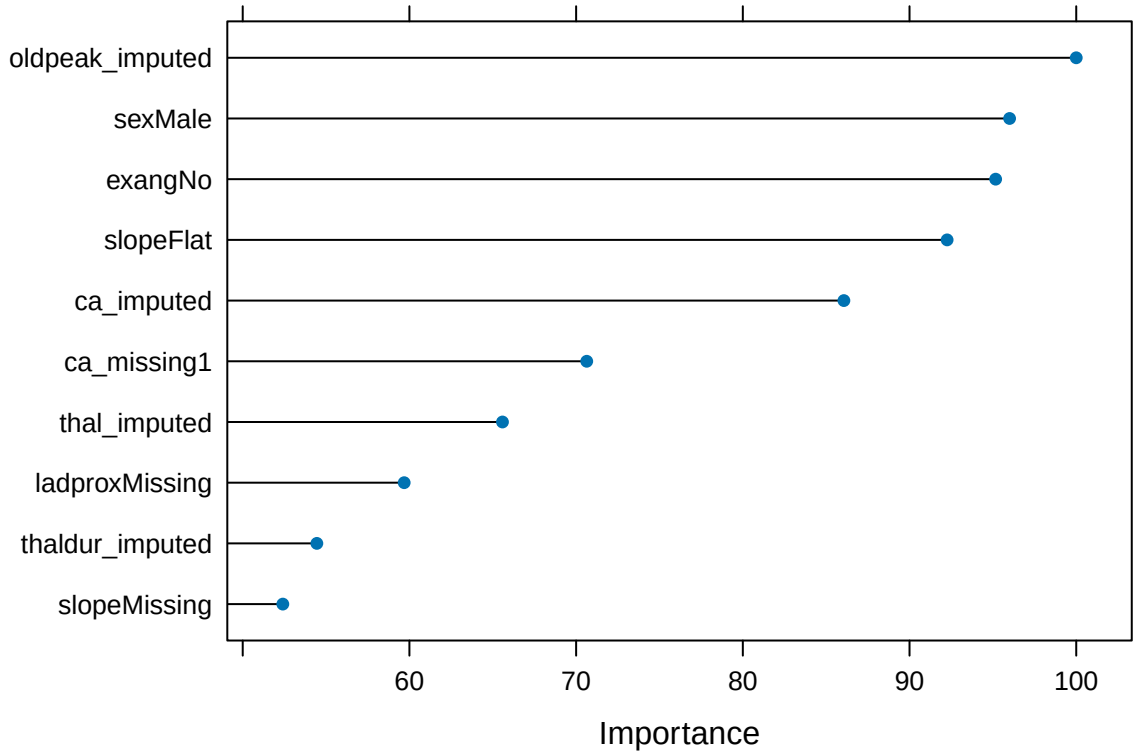
```r
glm_train_preds  <- predict(glm_model, train_data)
glm_train_probs  <- predict(glm_model, train_data, type = "prob")[, "Yes"]
roc_glm_train    <- roc(train_data$has_disease, glm_train_probs)
```

```
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
```

```r
glm_cm_train     <- confusionMatrix(glm_train_preds, train_data$has_disease,
                                    positive = "Yes")
# Feature Importance
glm_imp <- varImp(glm_model)
# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model       = "Logistic Regression",
    Accuracy    = glm_cm_test$overall["Accuracy"],
    Sensitivity = glm_cm_test$byClass["Sensitivity"],
    Specificity = glm_cm_test$byClass["Specificity"],
    ROC_AUC     = as.numeric(auc(roc_glm))
  )
# Display train/test AUC
cat("Logistic Regression - Train AUC:", as.numeric(auc(roc_glm_train)),
    "Test AUC:", as.numeric(auc(roc_glm)), "\n")
```

```
## Logistic Regression - Train AUC: 0.9592218 Test AUC: 0.8343434
```

```r
# Plot importance
plot(glm_imp, top = 10)
```



2. Random Forest

```
#set seed
set.seed(seed)
#train model
rf_model <- train(
  has_disease ~ .,
  data       = train_data,
  method     = "rf",
  metric     = global_metric,
  trControl  = ctrl,
  preProcess = process_strat,
  importance = TRUE
)

# Testing set performance
rf_preds      <- predict(rf_model, test_data)
rf_probs      <- predict(rf_model, test_data, type = "prob")[, "Yes"]
roc_rf        <- roc(test_data$has_disease, rf_probs)
```

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

```
rf_cm_test    <- confusionMatrix(rf_preds, test_data$has_disease,
                                 positive = "Yes")
```

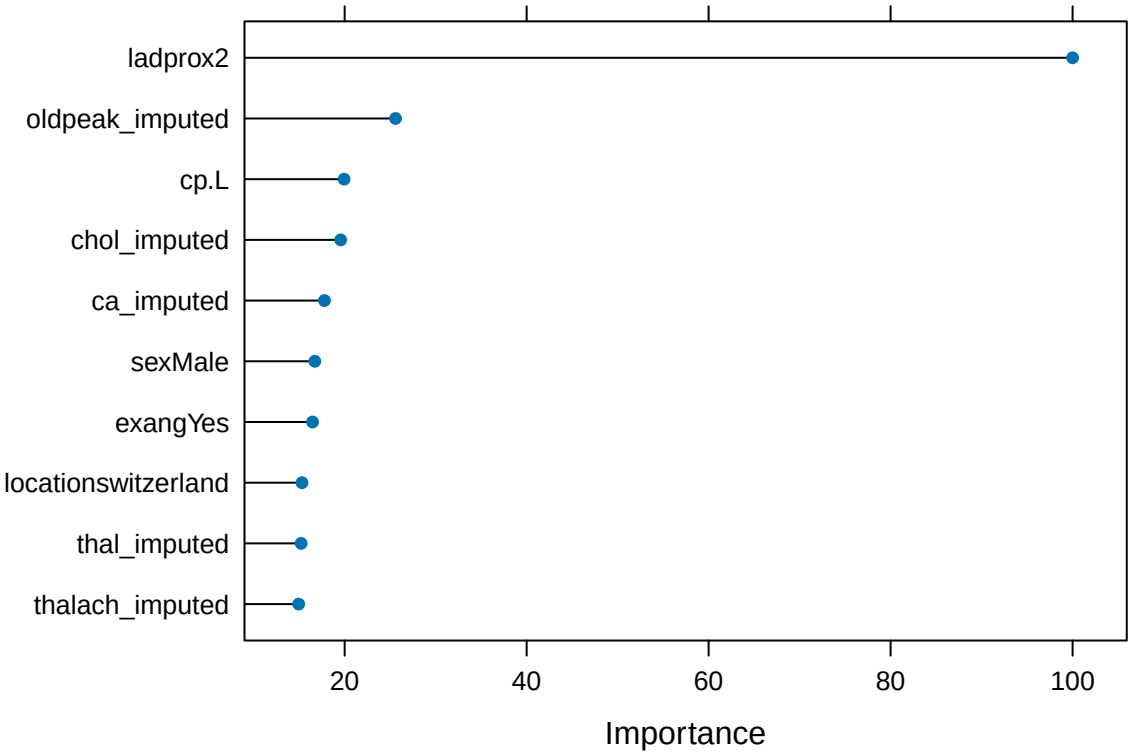
```r
# Train set performance
rf_train_preds <- predict(rf_model, train_data)
rf_train_probs <- predict(rf_model, train_data, type = "prob")[, "Yes"]
roc_rf_train   <- roc(train_data$has_disease, rf_train_probs)
```

```
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
```

```r
rf_cm_train    <- confusionMatrix(rf_train_preds, train_data$has_disease,
                                  positive = "Yes")
# Feature Importance
rf_imp <- varImp(rf_model)
# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model       = "Random Forest",
    Accuracy    = rf_cm_test$overall["Accuracy"],
    Sensitivity = rf_cm_test$byClass["Sensitivity"],
    Specificity = rf_cm_test$byClass["Specificity"],
    ROC_AUC     = as.numeric(auc(roc_rf))
  )
# Display train/test AUC
cat("Random Forest - Train AUC:", as.numeric(auc(roc_rf_train)),
    "Test AUC:", as.numeric(auc(roc_rf)), "\n")
```

```
## Random Forest - Train AUC: 1 Test AUC: 0.9066919
```

```r
# Plot importance
plot(rf_imp, top = 10)
```



3. PLS Discriminant Analysis

```
#set seeed
set.seed(seed)
#train model
pls_model <- train(
  has_disease ~ .,
  data       = train_data,
  method     = "pls",
  tuneGrid   = pls_grid,
  preProcess = process_strat,
  metric     = global_metric,
  trControl  = ctrl
)

# Testing set performance
pls_preds      <- predict(pls_model, test_data)
pls_probs      <- predict(pls_model, test_data, type = "prob")[, "Yes"]
roc_pls        <- roc(test_data$has_disease, pls_probs)
```

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

```
pls_cm_test    <- confusionMatrix(pls_preds, test_data$has_disease,
                                  positive = "Yes")
```

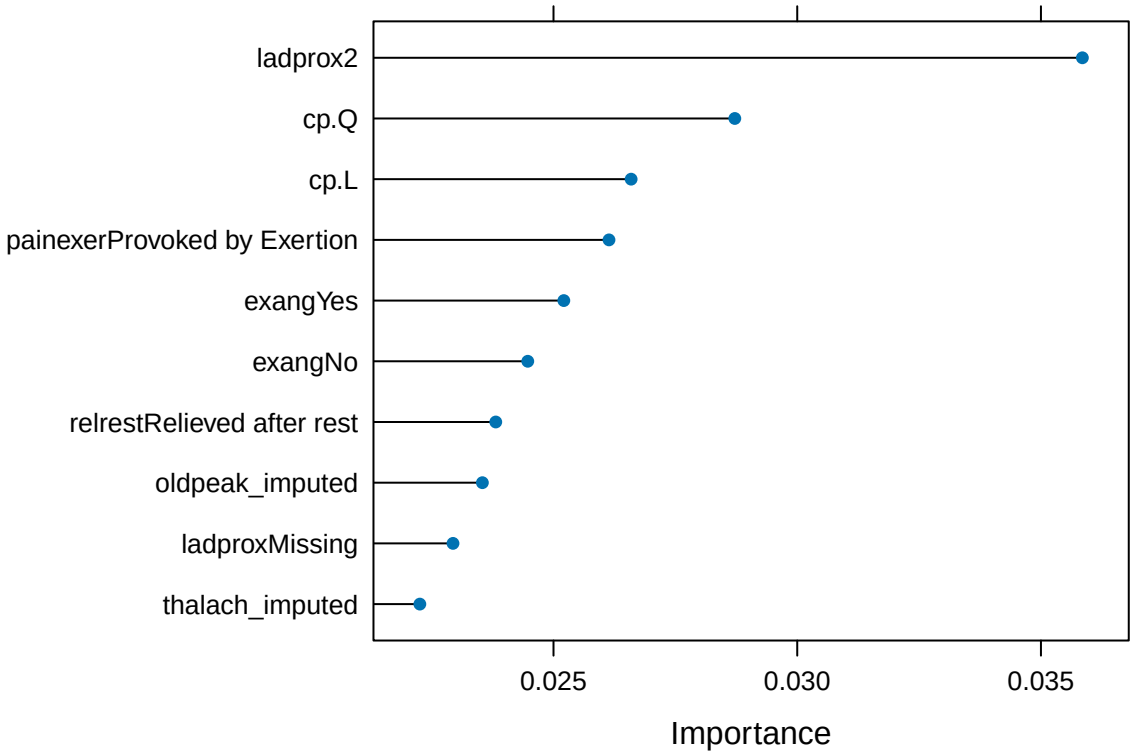
```r
# Training set performance
pls_train_preds <- predict(pls_model, train_data)
pls_train_probs <- predict(pls_model, train_data, type = "prob")[, "Yes"]
roc_pls_train   <- roc(train_data$has_disease, pls_train_probs)
```

```
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
```

```r
pls_cm_train    <- confusionMatrix(pls_train_preds, train_data$has_disease,
                                   positive = "Yes")
# Feature Importance
pls_imp <- varImp(pls_model, scale = FALSE)
# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model       = "PLS-DA",
    Accuracy    = pls_cm_test$overall["Accuracy"],
    Sensitivity = pls_cm_test$byClass["Sensitivity"],
    Specificity = pls_cm_test$byClass["Specificity"],
    ROC_AUC     = as.numeric(auc(roc_pls))
  )
# Display train/test AUC
cat("PLS-DA - Train AUC:", as.numeric(auc(roc_pls_train)),
    "Test AUC:", as.numeric(auc(roc_pls)), "\n")
```

```
## PLS-DA - Train AUC: 0.9441405 Test AUC: 0.9113636
```

```r
# Plot importance
plot(pls_imp, top = 10)
```



4. K-Nearest Neighbors

```
# set the seed
set.seed(seed)
#build model
knn_model <- train(
  has_disease ~ .,
  data       = train_data,
  method     = "knn",
  tuneGrid   = K_grid,
  metric     = global_metric,
  trControl  = ctrl,
  preProcess = process_strat
)

# Testing set performance
knn_preds     <- predict(knn_model, test_data)
knn_probs     <- predict(knn_model, test_data, type = "prob")[, "Yes"]
roc_knn       <- roc(test_data$has_disease, knn_probs)
```

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

```
knn_cm_test   <- confusionMatrix(knn_preds, test_data$has_disease,
                                 positive = "Yes")
```

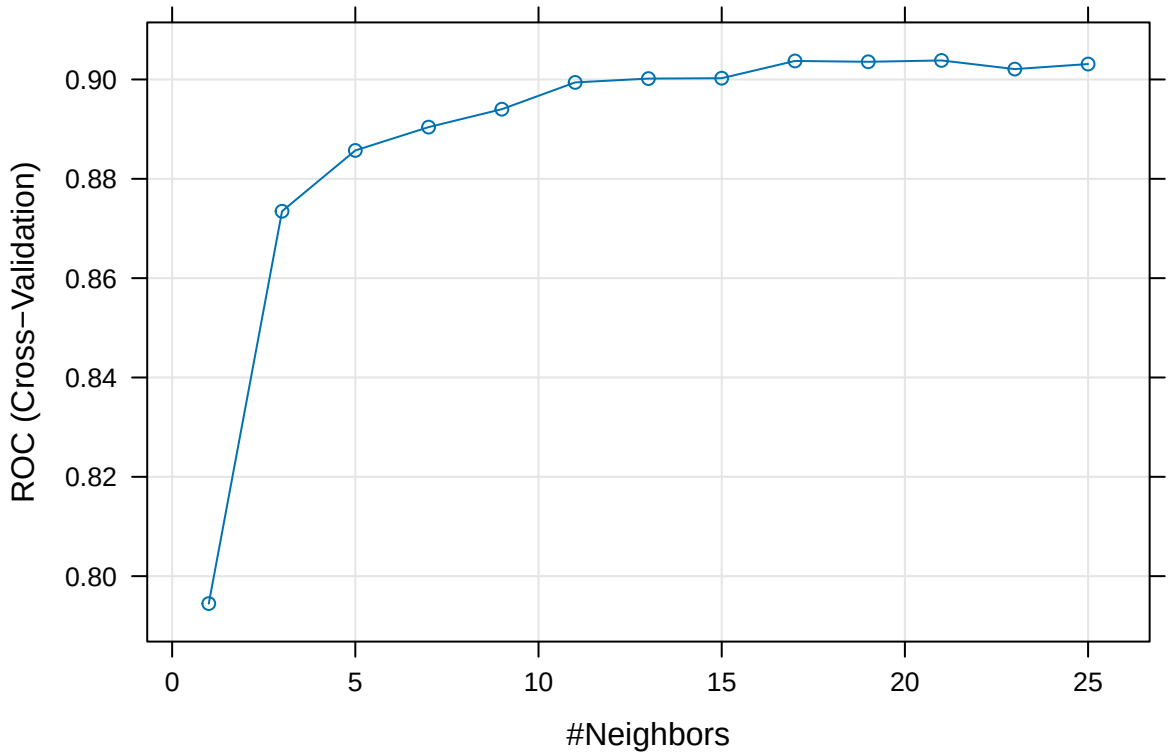
```r
# Training set performance
knn_train_preds <- predict(knn_model, train_data)
knn_train_probs <- predict(knn_model, train_data, type = "prob")[, "Yes"]
roc_knn_train   <- roc(train_data$has_disease, knn_train_probs)
```

```
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
```

```r
knn_cm_train    <- confusionMatrix(knn_train_preds, train_data$has_disease,
                                   positive = "Yes")
# Inspect KNN performance across K
cat("Best k:", knn_model$bestTune$k, "")
```

```
## Best k: 21
```

```r
plot(knn_model, metric = "ROC")
```




```r
# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model       = "K-Nearest Neighbors",
    Accuracy    = knn_cm_test$overall["Accuracy"],
    Sensitivity = knn_cm_test$byClass["Sensitivity"],
    Specificity = knn_cm_test$byClass["Specificity"],
    ROC_AUC     = as.numeric(auc(roc_knn))
  )
# Display train/test AUC
```

```r
cat("KNN - Train AUC:", as.numeric(auc(roc_knn_train)),
    "Test AUC:", as.numeric(auc(roc_knn)), "\n")
```

```
## KNN - Train AUC: 0.9257623 Test AUC: 0.9016414
```

5. Penalized Regression

```r
#set seed
set.seed(seed)
#train model
lasso_model <- train(
  has_disease ~ .,
  data       = train_data,
  method     = "glmnet",
  tuneGrid   = lasso_grid,
  metric     = global_metric,
  trControl  = ctrl,
  preProcess = process_strat
)
# best tuning parameters
lasso_best <- lasso_model$bestTune
# Testing set performance
lasso_preds   <- predict(lasso_model, test_data)
lasso_probs   <- predict(lasso_model, test_data, type = "prob")[, "Yes"]
roc_lasso     <- roc(test_data$has_disease, lasso_probs)
```

a. Lasso Penalization

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

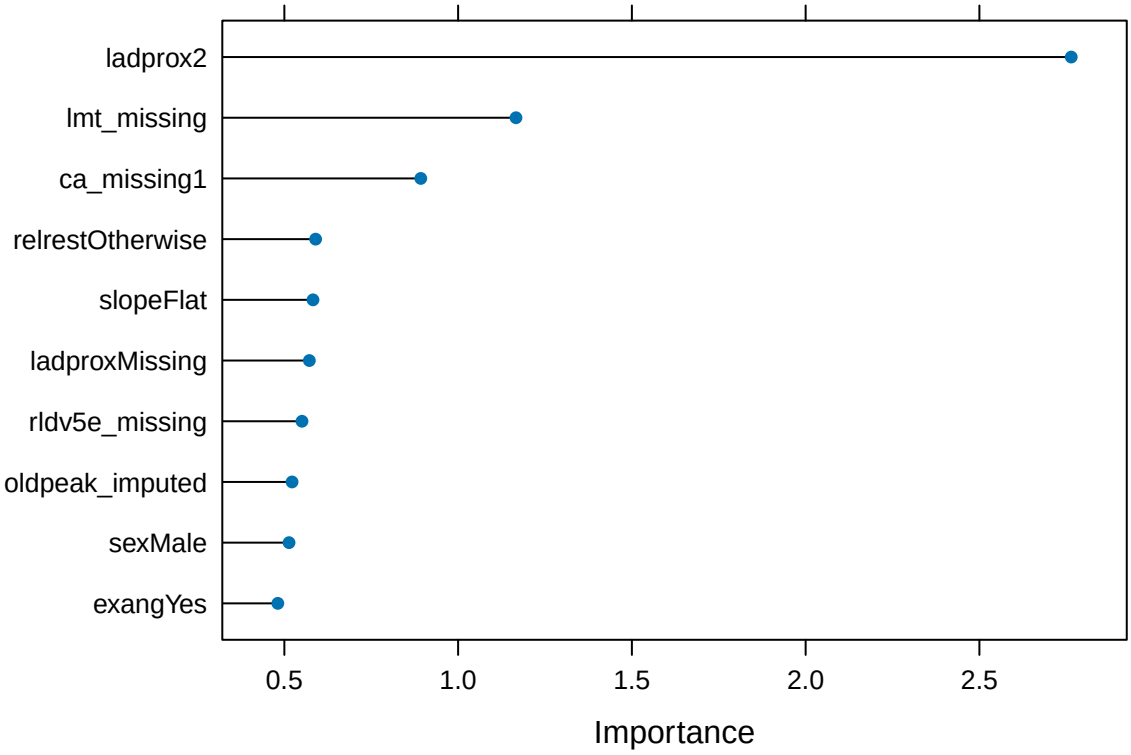
```r
lasso_cm      <- confusionMatrix(lasso_preds, test_data$has_disease,
                                 positive = "Yes")

# Training set performance
lasso_train_preds <- predict(lasso_model, train_data)
lasso_train_probs <- predict(lasso_model, train_data, type = "prob")[, "Yes"]
roc_lasso_train   <- roc(train_data$has_disease, lasso_train_probs)
```

```
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
```

```r
lasso_cm_train    <- confusionMatrix(lasso_train_preds, train_data$has_disease,
                                     positive = "Yes")
#lasso importance
lasso_imp <- varImp(lasso_model, scale = FALSE)
# append results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model       = "Lasso",
    Accuracy    = as.numeric(lasso_cm$overall["Accuracy"]),
    Sensitivity = as.numeric(lasso_cm$byClass["Sensitivity"]),
    Specificity = as.numeric(lasso_cm$byClass["Specificity"]),
    ROC_AUC     = as.numeric(auc(roc_lasso))
```

```
  )
# Plot importance
plot(lasso_imp, top = 10)
```




```
#set seed
set.seed(seed)
# build model
ridge_model <- train(
  has_disease ~ .,
  data       = train_data,
  method     = "glmnet",
  tuneGrid   = ridge_grid,
  metric     = global_metric,
  trControl  = ctrl,
  preProcess = process_strat
)
#best tuning parameters
ridge_best <- ridge_model$bestTune
# Testing set performance
ridge_preds <- predict(ridge_model, test_data)
ridge_probs <- predict(ridge_model, test_data, type = "prob")[, "Yes"]
roc_ridge   <- roc(test_data$has_disease, ridge_probs)
```

b. Ridge Penalization

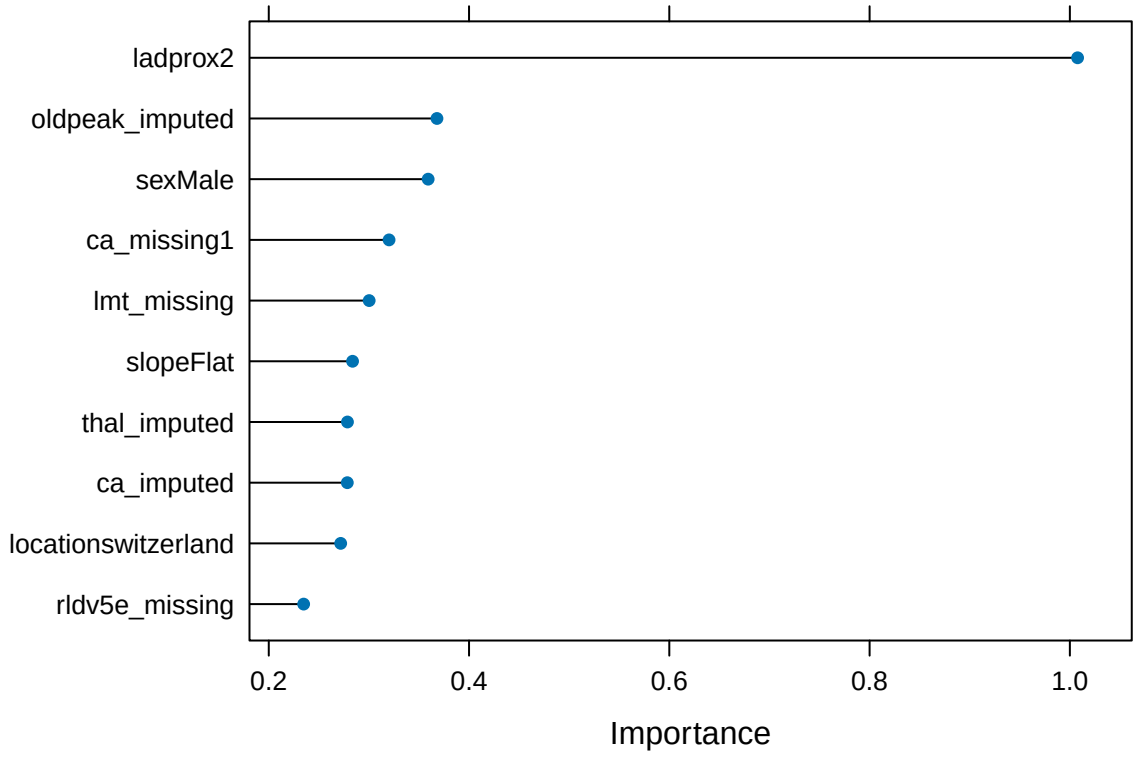
```
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
```

```
ridge_cm    <- confusionMatrix(ridge_preds, test_data$has_disease,
                               positive = "Yes")

# training set performance
ridge_train_preds <- predict(ridge_model, train_data)
ridge_train_probs <- predict(ridge_model, train_data, type = "prob")[, "Yes"]
roc_ridge_train   <- roc(train_data$has_disease, ridge_train_probs)
```

```
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
```

```
ridge_cm_train    <- confusionMatrix(ridge_train_preds, train_data$has_disease,
                                     positive = "Yes")
# Feature Importance
ridge_imp <- varImp(ridge_model, scale = FALSE)
# Append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model       = "Ridge",
    Accuracy    = ridge_cm$overall["Accuracy"],
    Sensitivity = ridge_cm$byClass["Sensitivity"],
    Specificity = ridge_cm$byClass["Specificity"],
    ROC_AUC     = as.numeric(auc(roc_ridge))
  )
# Plot importance
plot(ridge_imp, top = 10)
```




```r
#set seed
set.seed(seed)
# train model
enet_model <- train(
  has_disease ~ .,
  data       = train_data,
  method     = "glmnet",
  tuneGrid   = enet_grid,
  metric     = global_metric,
  trControl  = ctrl,
  preProcess = process_strat
)
# best tuning parameters
enet_best <- enet_model$bestTune
# Testing set performance
enet_preds <- predict(enet_model, test_data)
enet_probs <- predict(enet_model, test_data, type = "prob")[, "Yes"]
roc_enet   <- roc(test_data$has_disease, enet_probs)
```

c. Elastic Net Model

```
## Setting levels: control = No, case = Yes
```

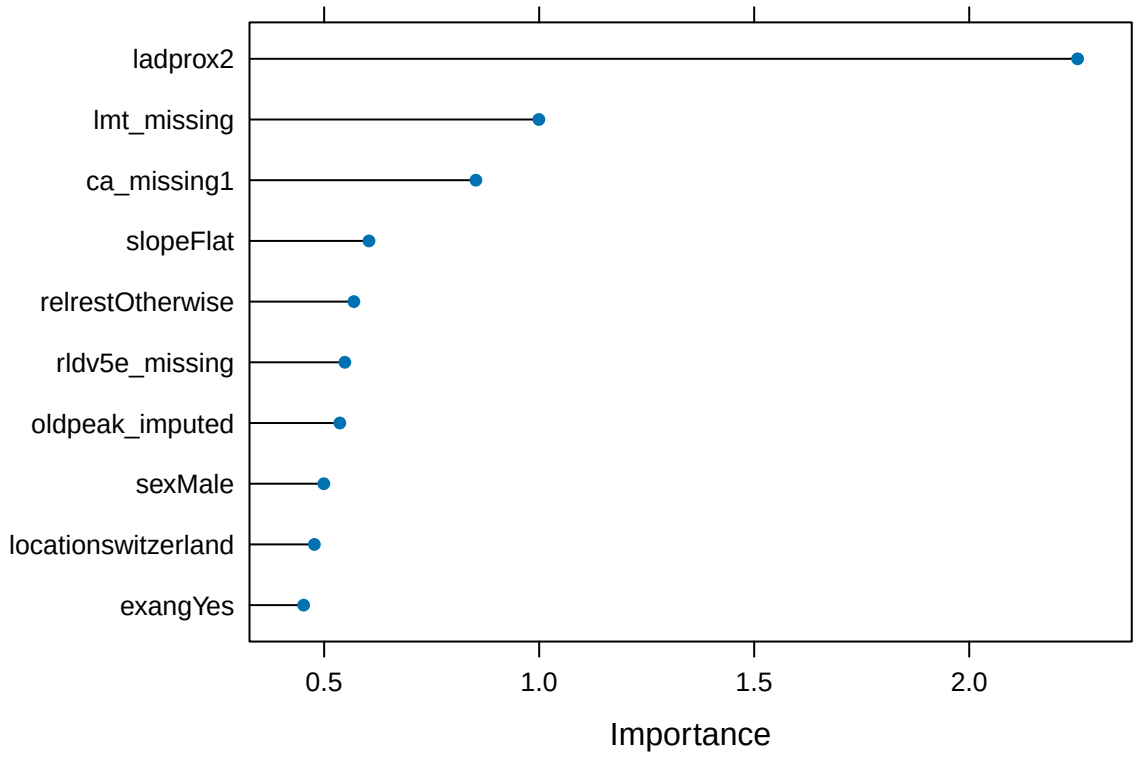
```
## Setting direction: controls < cases
```

```r
enet_cm    <- confusionMatrix(enet_preds, test_data$has_disease,
                              positive = "Yes")

# Training set performance
enet_train_preds <- predict(enet_model, train_data)
enet_train_probs <- predict(enet_model, train_data, type = "prob")[, "Yes"]
roc_enet_train   <- roc(train_data$has_disease, enet_train_probs)
```

```
## Setting levels: control = No, case = Yes
## Setting direction: controls < cases
```

```r
enet_cm_train    <- confusionMatrix(enet_train_preds, train_data$has_disease,
                                    positive = "Yes")
# enet imp
enet_imp <- varImp(enet_model, scale = FALSE)
# append to results_tbl
results_tbl <- results_tbl %>%
  add_row(
    Model       = "Elastic Net",
    Accuracy    = enet_cm$overall["Accuracy"],
    Sensitivity = enet_cm$byClass["Sensitivity"],
    Specificity = enet_cm$byClass["Specificity"],
    ROC_AUC     = as.numeric(auc(roc_enet))
  )
#plot important features
plot(enet_imp, top = 10)
```



Results Summary and ROC Comparison

```
# ROC results comparison
knitr::kable(results_tbl, digits = 3, caption = "Model Performance Comparison")
```

Table 1: Model Performance Comparison

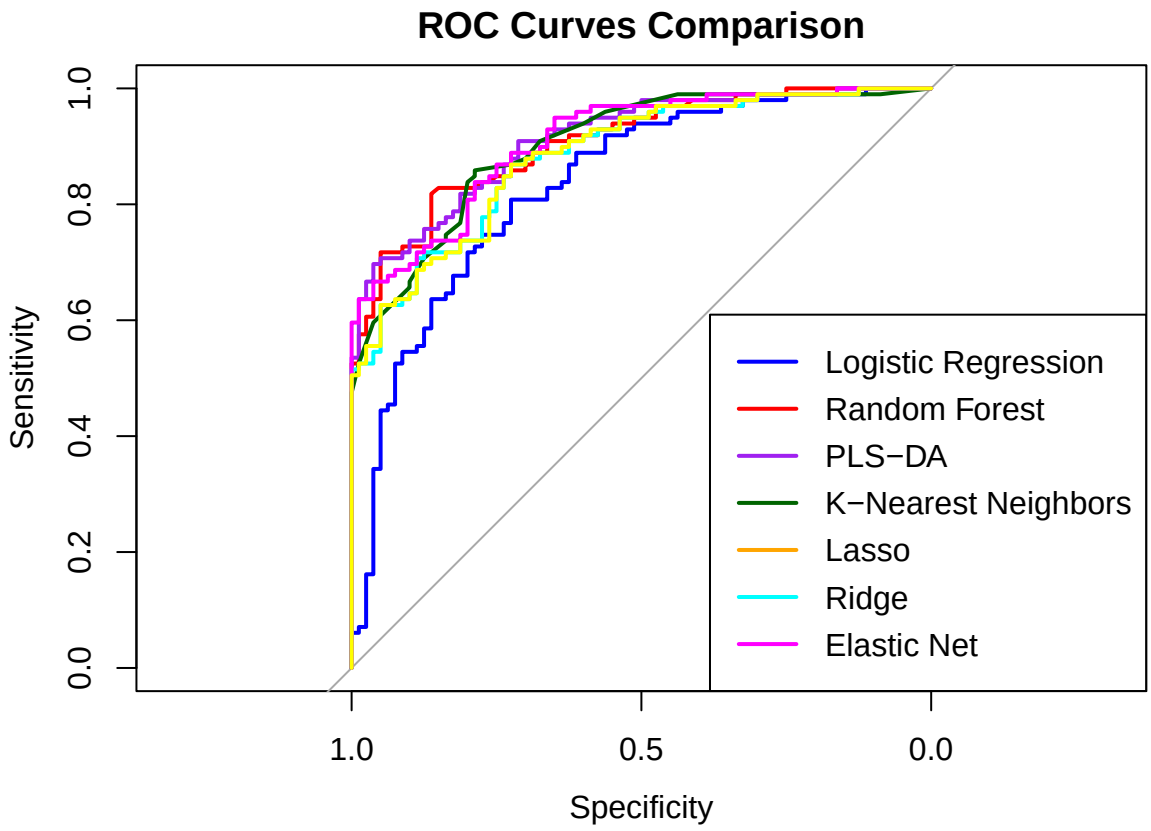
Model	Accuracy	Sensitivity	Specificity	ROC_AUC
Logistic Regression	0.749	0.727	0.775	0.834
Random Forest	0.821	0.788	0.863	0.907
PLS-DA	0.804	0.798	0.812	0.911
K-Nearest Neighbors	0.821	0.848	0.787	0.902
Lasso	0.760	0.737	0.787	0.884
Ridge	0.799	0.808	0.787	0.909
Elastic Net	0.760	0.737	0.787	0.884

```
plot(roc_glm,   col = "blue",      lwd = 2, main = "ROC Curves Comparison")
lines(roc_rf,   col = "red",       lwd = 2)
lines(roc_pls,  col = "purple",    lwd = 2)
lines(roc_knn,  col = "darkgreen", lwd = 2)
lines(roc_lasso, col = "cyan",     lwd = 2)
lines(roc_ridge, col = "magenta",  lwd = 2)
lines(roc_enet, col = "yellow",    lwd = 2)

legend("bottomright",
       legend = results_tbl$Model,
       col    = c("blue","red","purple","darkgreen","orange","cyan","magenta","yellow"),
       lwd    = 2)
```

Model Performance Comparison

Model	Accuracy	Sensitivity	Specificity	ROC_AUC
Logistic Regression	0.749	0.727	0.775	0.834
Random Forest	0.821	0.788	0.863	0.907
PLS-DA	0.804	0.798	0.812	0.911
K-Nearest Neighbors	0.821	0.848	0.787	0.902
Lasso	0.760	0.737	0.787	0.884
Ridge	0.799	0.808	0.787	0.909
Elastic Net	0.760	0.737	0.787	0.884




```
#results_tbl with gt()
results_tbl %>%
  gt() %>%
  fmt_number(
    columns = c(Accuracy, Sensitivity, Specificity, ROC_AUC),
    decimals = 3) %>%
  tab_header(title = "Model Performance Comparison")
```